

Machine Learning for Mechatronics

Module 2: Machine Learning Techniques

Lecture 1: Introduction and Overview

Dr. Rajan Prasad

SMEC
VIT University

Fall 2026



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

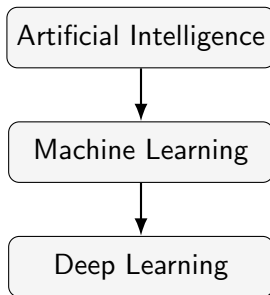
Traditional mechatronic systems depend on:

- mathematical models,
- fixed rules,
- manually tuned controllers.

Machine Learning is useful when systems involve:

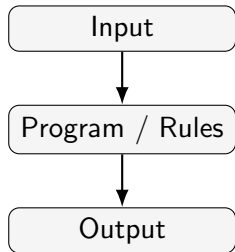
- nonlinear behaviour,
- uncertain environments,
- large sensor data,
- adaptive decision-making.

Examples: autonomous robots, predictive maintenance, smart manufacturing, intelligent vehicles.

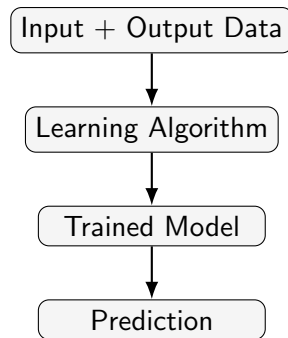


AI	ML	Deep Learning
Broad field	Learns from data	Neural networks
Rules + reasoning	Prediction models	Feature learning
Expert systems	Regression/classification	CNN, RNN, etc.

Traditional Programming



Machine Learning



Supervised Learning

- Uses labelled data
- Regression
- Classification

Example: motor fault classification

Unsupervised Learning

- No labelled output
- Clustering
- Dimensionality reduction

Example: sensor pattern grouping

Reinforcement Learning

- Agent interacts with environment
- Learns from rewards
- Used for control decisions

Example: robot navigation

Regression (Predicting Continuous Values)

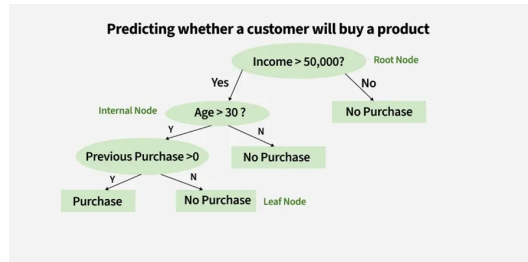
- Maps inputs to a continuous, numerical value.
- Find a line or curve of best fit that minimizes the error (e.g., Mean Squared Error)
- Soft Sensing. Estimating the real-time internal temperature of an electric motor rotor using accessible surface data

Classification (Predicting Discrete Categories)

- Maps inputs to a discrete label, class, or category (e.g., Yes/No, Healthy/Faulty)
- Construct an optimal decision boundary (a hyperplane) that clearly separates different classes of data.
- Automated Fault Diagnosis: Automated Defect Detection (or Quality Inspection) on a fast-moving production line

Decision Trees

- Partitions data regions through a sequence of recursive, conditional splits.
- Can be used for Classification and/or Regression
- Works like a flowchart that helps in making step-by-step decisions
- Embedded mechatronics: *if – else* loops in C/C++.

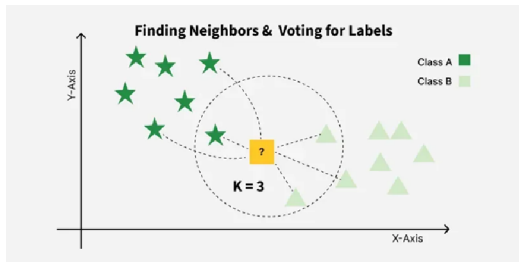


Reference: <https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/>

k-Nearest Neighbors (k-NN)

- Instance-based (lazy learning) algorithm
- Can be used for Classification and/or Regression
- Identifies the k closest data points (neighbors) and computes the prediction.
- Choose k , Calculate Distance (eg. Euclidean Distance), Find Neighbors, Predict

Example: Daily delivery addresses sorting

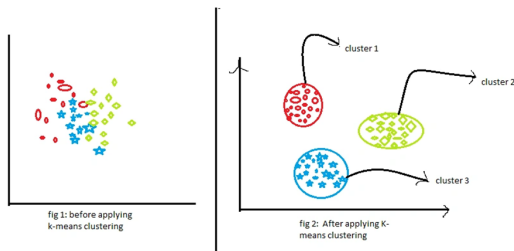


Reference: <https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>

Clustering

k-Means Clustering

- Groups unlabeled data into k distinct categories based on their similarities.
- Operates by iteratively measuring the distance from data points to central points (centroids) to minimize the variation within each cluster
- **Customer Segmentation:** Grouping buyers with similar purchasing habits for targeted marketing, **Anomaly Detection:** Identifying outliers that do not fall neatly into any



Reference: <https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/>

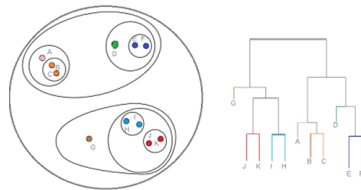
Clustering Hierarchical Clustering

- Groups data into a nested tree of clusters (Hierarchy)
- Builds a tree-like structure (dendrogram) that helps visualize relationships and decide the optimal number of clusters

Two main types

- Agglomerative clustering [bottom-up approach]
- Divisive clustering [top-down approach]

Example: Song Categorization: Organize millions of songs to genres, sub-genres, and micro-genres without manually human tag.

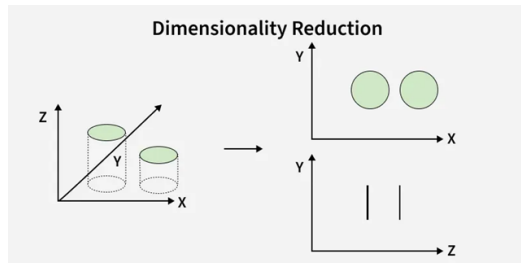


Reference: <https://www.statisticshowto.com/hierarchical-clustering/>

[//www.statisticshowto.com/hierarchical-clustering/](https://www.statisticshowto.com/hierarchical-clustering/)

Dimensionality Reduction

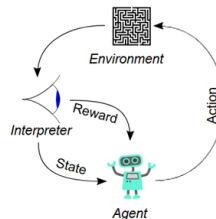
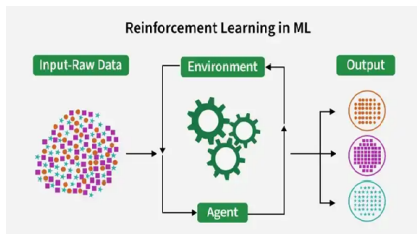
- Technique used to reduce the number of features in a dataset while preserving important information.
- Transforms high-dimensional data into a lower-dimensional space for simpler representation.
- Two main steps
- Feature Selection (Filtering, Wrapper, Missing value ratio etc..)
- Feature Extraction (Principal Component Analysis (PCA), Independent Component Analysis (ICA), etc..)



Reference: <https://www.geeksforgeeks.org/machine-learning/dimensionality-reduction/>

Focuses on how agents learn to make decisions through trial and error to maximize cumulative rewards.

- The agent interacts with an environment and receives feedback based on its actions.

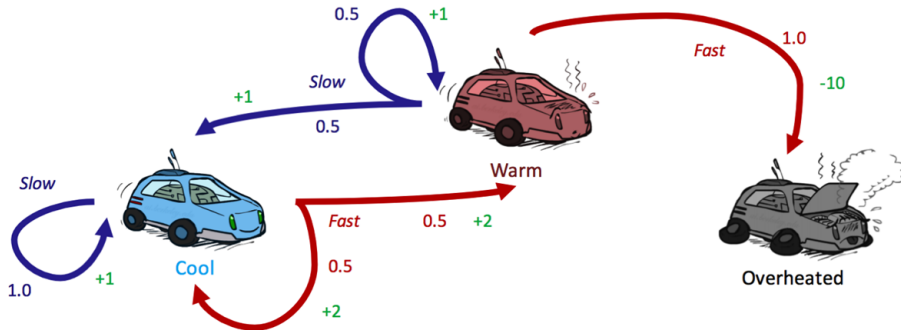


Reference: <https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/>

- **Agent:** The decision-maker.
- **Environment:** The system in which the agent operates.
- **State:** The current condition of the system.
- **Action:** A possible decision or move.
- **Reward:** Feedback received after an action.

Markov Decision Process

- Mathematical model for sequential decision making when outcomes are uncertain



Reference:

<https://inst.eecs.berkeley.edu/~cs188/textbook/mdp/markov-decision-processes.html>

Markov Decision Process

concepts of MDP- form of a tuple (S, A, p, r) :

States (S): A state of space in the system $\{cool, warm, overheated\}$

Actions (A): A set of actions $\{slow, fast\}$

Transition probabilities (p): The probability of state transitions $T(S, A, S')$

- $T(slow, cool, slow) = 1$
- $T(warm, slow, cool) = 0.5$

Reward function (r): Cost or reward of performing an action in a specific state $R(S, A, S')$

- $R(slow, cool, slow) = 1$
- $R(warm, fast, overheated) = -10$

Example

- **Video Games**: MDPs help characters to choose the best moves to win or complete tasks even when outcomes are not certain.
- **Robots**: MDPs to decide how to move safely and efficiently in places like factories or warehouses and avoid obstacles.

Example: Mobile Robot Navigation

State

$$S = (x, y, \theta)$$

Action

$$A = \{Forward, Left, Right\}$$

Transition

$$T(S'|S, A)$$

Reward

$$R = \begin{cases} +100, & \text{Goal} \\ -100, & \text{Collision} \\ -1, & \text{Otherwise} \end{cases}$$

S			
			G

Q-Learning

- Q-Learning is a **model-free Reinforcement Learning algorithm**.
- It learns the best action for every state through trial and error.
- The agent does not require prior knowledge of the environment.
- It improves its decisions by maximizing the cumulative reward.

Key Characteristics

- Model-free
- Value-based
- Off-policy
- Learns optimal behaviour through experience

Objective

Robot must reach the charging station.

- Start position (S)
- Goal (G)
- Avoid obstacle (X)
- Learn shortest path

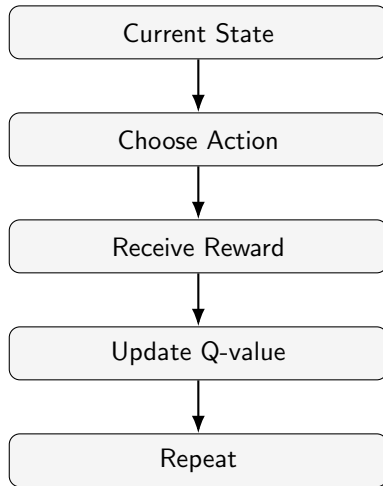
S		
	X	
		G

Rewards

- Goal = +100
- Obstacle = -100
- Every movement = -1

Example:

How can the robot learn the best path without being programmed?



$$Q(s, a) = Q(s, a) + \alpha [R + \gamma \max Q(s', a) - Q(s, a)]$$

Meaning of the Terms

- $Q(s, a)$: Current estimate
- α : Learning rate
- R : Immediate reward
- γ : Discount factor
- $\max Q(s', a)$: Best future reward

What is a Q-value?

- Q-value estimates how good an action is.
- Higher Q-value means better action.
- Robot stores Q-values in a table.

State	Up	Down	Left	Right
S0	0	0	0	0
S1	2	5	1	15
S2	8	12	4	25
S3	10	8	6	30

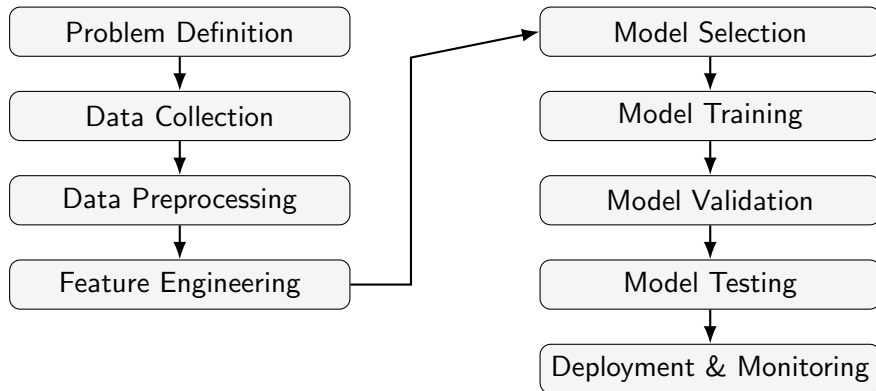
Initially all values are zero. They improve after repeated learning.

Application	Example
Mobile Robots	Path planning
Quadruped Robots	Footstep optimization
Industrial Robots	Pick-and-place sequencing
AGVs	Warehouse navigation
Underwater Robots	Obstacle avoidance
Drones	Autonomous flight
Smart Manufacturing	Adaptive process control

Example [Biological Analogy]:

- I have recently joined VIT. Going to Guesthouse from Homeland. Q-value table based on time taken in different route.
- NASA training of chimpanzees for the Project Mercury space program in the late 1950s and early 1960s.

Aspect	Markov Decision Process (MDP)	Q-Learning
Purpose	Mathematical framework for sequential decision making	Algorithm that learns the optimal policy
Knowledge Required	Transition probabilities and reward function are assumed known	Transition probabilities are not required
Type	Environment model	Learning algorithm
Model	Model-based	Model-free
Learning	Does not learn	Learns through interaction
Output	Optimal policy (using DP methods)	Optimal Q-table and policy
Applications	Control theory, planning	Robotics, games, autonomous systems



Model Selection is the process of choosing the most appropriate machine learning algorithm for a given problem.

Selection Criteria

- Type of problem
 - Regression
 - Classification
 - Clustering
- Dataset size
- Feature dimension
- Computational complexity
- Model interpretability
- Prediction accuracy

Problem	Model
Regression	Linear Regression
Classification	Logistic Regression
Classification	SVM
Clustering	K-Means
Images	CNN
Sequences	LSTM
Decision Making	Q-Learning

Model Training is the process of learning the model parameters from the training data.

Objective

Find model parameters that minimize the prediction error.

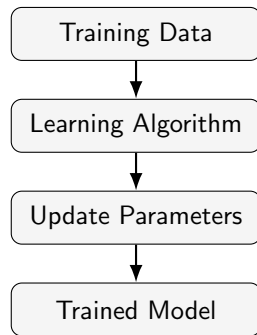
$$\theta^* = \arg \min_{\theta} L(\theta)$$

Example (Linear Regression)

$$y = \theta_0 + \theta_1 x$$

Loss Function (Mean Squared Error)

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$



Dataset Split

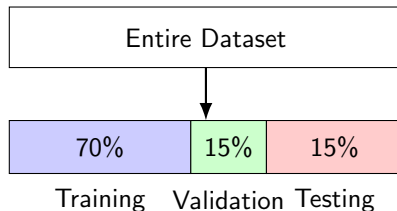
- Training Set (70%)
- Validation Set (15%)
- Testing Set (15%)

Validation

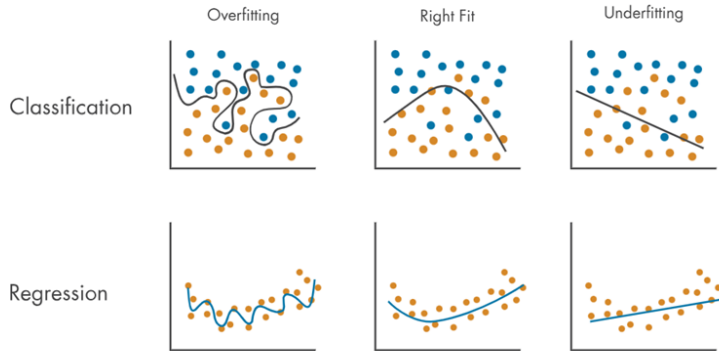
- Tune hyperparameters
- Compare different models
- Detect overfitting

Testing

- Evaluate the final model
- Estimate generalization performance
- Performed only once after model selection



Underfitting vs Overfitting



Example:

Reference: <https://uk.mathworks.com/discovery/overfitting.html>

Underfitting

- Model is too simple.
- Cannot capture underlying patterns.
- High training error.
- High testing error.
- High Bias, Low Variance.

Example

Linear model for nonlinear robot dynamics.

Goal: Find a model that generalizes well to unseen data.

Refer to : Python code file [AI_For_Mechatronics_Course.ipynb]

Overfitting

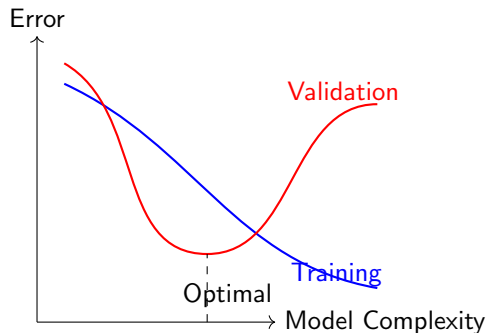
- Model is too complex.
- Learns noise in the training data.
- Very low training error.
- High testing error.
- Low Bias, High Variance.

Example

Decision tree with excessive depth.

Observation

- Training error decreases continuously.
- Validation error decreases initially.
- After a certain point, validation error increases.
- The increase indicates overfitting.



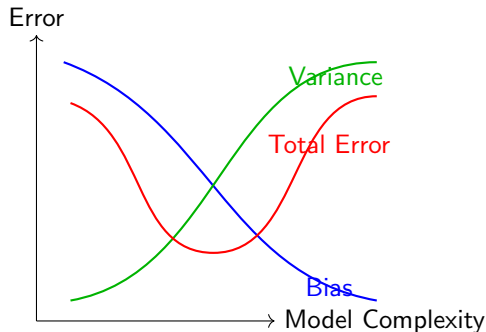
Model selection should be performed at the minimum validation error.

Bias

- Error due to overly simple assumptions.
- Causes underfitting.
- High Bias $\Rightarrow$ Low model complexity.

Variance

- Error due to sensitivity to training data.
- Causes overfitting.
- High Variance $\Rightarrow$ High model complexity.



Best model complexity occurs at the minimum of the total error curve.

Problem

Predict the health of an electric motor using:

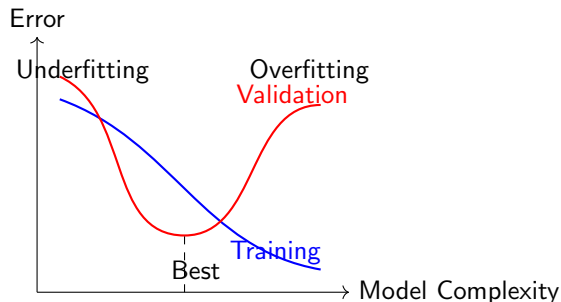
Vibration, Temperature, Current, Speed

Three models are trained.

Model	Train	Test
Linear Regression	72%	70%
Decision Tree	100%	78%
Random Forest	95%	93%

Interpretation

- Linear Regression → Underfitting
- Decision Tree → Overfitting
- RL → Best Generalization



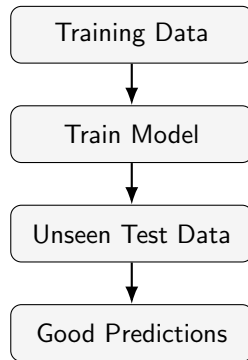
Conclusion

A machine learning engineer does not choose the model with the **highest training accuracy**. They choose the model that performs best on **unseen validation or test data**.

Model generalization is the ability of a trained machine learning model to make accurate predictions on **previously unseen data** rather than simply memorizing the training data.

Characteristics of a Good Generalized Model

- Low training error
- Low validation/testing error
- Learns underlying patterns
- Robust to noisy inputs
- Performs well in real-world applications



Goal: A model should learn the **true relationship** between inputs and outputs, not the noise present in the training data.